

STUDENT CASE STUDY 5: DASHBOARD COMPLETION, REPORTING, & ALERT SYSTEM

Course: CMU-CS 445 – *System Integration Practices*

Semester: 2

Academic: 2024–2025

Instructor: MSc. Nguyen Dang Quang Huy

1. Objective

The purpose of this case study is to complete the **system integration project** by connecting the **frontend dashboard** with the **secured backend APIs** developed in previous case studies.

Students will integrate real data through RESTful APIs, visualize reports, implement smart alerts, and finalize the user interface with proper role-based security.

This stage demonstrates the **final implementation phase** of the **System Integration Life Cycle (SILC)**: focusing on data visualization, monitoring, and end-user accessibility.

2. Scenario Background

After developing backend APIs and security modules in Case Studies 3 and 4, Company X requires the completion of an interactive **Dashboard Application** that provides:

- Real-time visibility of HR and payroll data.
- Dynamic reporting and analytics features.
- Automated alerts for HR events and payroll anomalies.
- Role-based data access with a secure and responsive interface.

Students will integrate all components into a single functioning system that can support organizational decision-making and performance monitoring.

3. Tasks and Requirements

3.1 API Integration into the Dashboard

The frontend must be developed using **React** or **ASP.NET MVC**, integrated with backend APIs.

Functional requirements:

- Fetch and display employee data from /employees.
- Fetch and display payroll data from /payroll.
- Implement search, filter, and pagination for employees and payroll.
- Develop forms to **add, update, and delete** employees, salary, and attendance records.
- Use **real API calls** with authentication tokens (JWT).
- Handle loading states, errors, and success notifications gracefully.

Technical guidelines:

- Use axios (React) or HttpClient (.NET) for API communication.
- Include authorization headers in all requests.
- Display response messages clearly to users.

3.2 Employee Management

- Display employee list retrieved from HUMAN_2025 via API.
- Implement search by Employee ID, Name, Department, or Job Title.
- Allow creation, modification, and deletion of employee data through API calls.
- Synchronize changes between HUMAN_2025 and PAYROLL automatically (based on backend logic).
- Validate input fields before submission (e.g., name, department, salary fields).
- Display confirmation dialogs for critical actions (delete/update).

3.3 Payroll and Attendance Management

- Display payroll details fetched from PAYROLL via /payroll API.
- View salary history per employee by month or year.
- Display attendance data (working days, absences, and leave days).
- Enable HR or Payroll Managers to update salary, bonuses, or deductions.
- Synchronize updates immediately across both systems.
- Protect payroll-related actions with appropriate authorization (only Payroll Manager or Admin).

3.4 Reporting and Analytics

- Implement **dynamic reports** using data retrieved from both HUMAN and PAYROLL.
- Reports to include:
 - HR Summary: number of employees by department and employment status.
 - Payroll Summary: total monthly payroll and salary distribution.
 - Shareholder Profit Report: profit allocation by quarter/year.
- Visualize data using charts and graphs (e.g., Chart.js, Recharts, or ASP.NET Chart Control).
- Include export functions (PDF, Excel, CSV) for management reporting.

3.5 Alert and Notification System

Develop a **Smart Alert Module** that continuously monitors key HR and payroll events and notifies relevant users.

Functional requirements:

- Work Anniversary Alerts: display employees reaching 1-year, 5-year, or 10-year milestones.
- Leave Alerts: notify HR when employees exceed allowed leave days.
- Payroll Discrepancy Alerts: detect significant salary changes between months.
- Email Notifications: send automated monthly salary statements to employees via SMTP.

Implementation guidelines:

- Backend triggers alerts via /alerts API.
- Frontend displays alerts in a notification panel or dashboard widget.
- Use **Nodemailer** (Node.js) or **.NET MailKit** for email delivery.
- Store alert logs for traceability.

3.6 User Interface and Experience Enhancement

- Improve UI/UX consistency and visual appeal.
- Use responsive design principles (e.g., Bootstrap or TailwindCSS).

- Optimize API calls for faster load time (debounce, caching).
- Display role-based interface:
 - Admin: Full system access.
 - HR Manager: HR-related sections only.
 - Payroll Manager: Payroll-related sections only.
 - Employee: Personal dashboard with limited access.
- Ensure accessibility and mobile-friendly design.

4. Expected Deliverables

4.1 Document Deliverables

Document Name	Action (New / Update)	Detailed Description / Student Instructions
18. Dashboard Implementation Report	New	Connect the Dashboard frontend (React or ASP.NET MVC) with your backend APIs. Each screen should now display real data. Explain how data is loaded, filtered, and updated. Example: Employee list from /employees, Payroll data from /payroll.
19. System Testing & Validation Report	New	Describe the results of full system testing. Include API integration, role-based access, data synchronization, and Dashboard responsiveness. Provide screenshots of successful API calls and UI displays. Summarize testing outcomes in tables.
20. Updated UI/UX Design Document (Final)	Update from Case 2	Replace mockups with real interface screenshots from your final system. Annotate key elements (buttons, search box, alert notifications). Describe improvements made based on user or instructor feedback.

21. Final Software Requirement Specification (SRS – Final Version)	Update from Case 4	Combine all sections (functional, integration, and security) into one final version. This document represents your full technical design and implementation record.
22. Final Project Report	New (Final Submission)	Write a professional summary report including: project overview, objectives, technologies, architecture, key challenges, solutions, and testing results. Conclude with what your team learned about system integration and future improvement ideas. Attach screenshots and references.

6.2 Software Functional Deliverables

Software Functions to Develop	Description / Implementation Guidance
1. Dashboard Integration (Frontend + API)	Connect Dashboard with real backend APIs. Replace mock data with live data from HUMAN and PAYROLL. Each section should display real values: employee list, payroll table, attendance summary, alerts.
2. Employee Management (Full)	Full CRUD integration: add, update, delete employees through API calls. Confirm data synchronization in both databases. Display success/error messages in UI.
3. Payroll & Attendance (Full)	Show real payroll data per employee. Allow updates to salary and bonuses. Display attendance data from PAYROLL and link to each employee.
4. Reports & Analytics (Dynamic)	Generate summary charts: • Total employees by department. • Payroll distribution per month.

	• Profit or expense overview (if data available). Use charts (Chart.js, Recharts, etc.).
5. Smart Alert System (Automated)	Implement backend logic to detect events: • Employees reaching 1/5/10-year milestones. • Employees exceeding leave limit. • Salary anomalies between periods. Send email or on-screen notifications.
6. Security Enforcement on Dashboard	Integrate JWT authentication into frontend. Redirect users based on role: • HR Manager: Employee pages. - Payroll Manager: Payroll pages. • Employee: personal info only.
7. System Testing & Final Deployment	Conduct full testing (API + UI). Ensure data integrity, access control, and system performance. Prepare a working demo version or video presentation showing integration in action.

5. Evaluation Criteria

Criteria	Description	Weight
API Integration	Proper API usage, secure communication, and error handling	20%
Functional Accuracy	Correct implementation of HR, payroll, and reporting features	25%
Alert & Notification System	Completeness, accuracy, and automation of alerts	20%
UI/UX Design	Clarity, responsiveness, and role-based interface	15%
Documentation & Presentation	Clarity, technical completeness, and professionalism	20%